

Overview

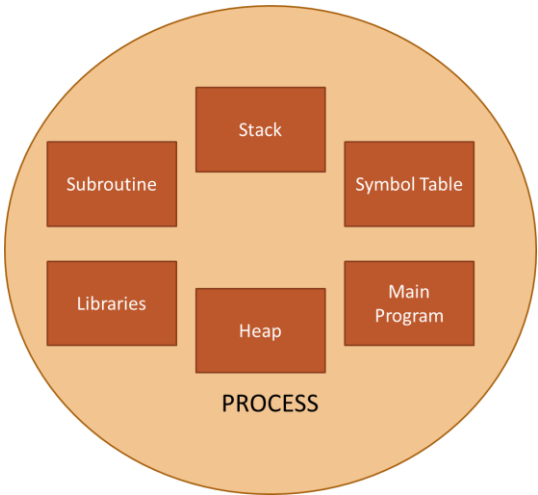
- How processes must be represented in code.
- Process scheduling is the heart of any operating system.
- Almost all platforms would have multiple processes running at the same time, requesting certain resources.
- The goal of the OS is to ensure that all processes execute, keeping the compute units (CPU and GPU) busy, while keeping the overall hardware stable.

Before we dive deep into CPU scheduling, we need to formalize the definition of a **process**, as these are the ones that will use the CPU.

INTRODUCTION TO PROCESSES

A process is a "**program in execution.**" It is more than just the executable code; it includes current activity, represented by the value of the program counter and the contents of the processor's registers.

- One program = one process.
- A standalone application that contains memory, stack, heap, etc.
- In C++, we use fork() method to create a duplicate process of the program.
- Context switch between the process is time consuming.
- We don't typically practice developing programs using multiple processes.
- Since we are simply going to emulate processes in the future, we represent processes as threads in the CSOPESY emulator.

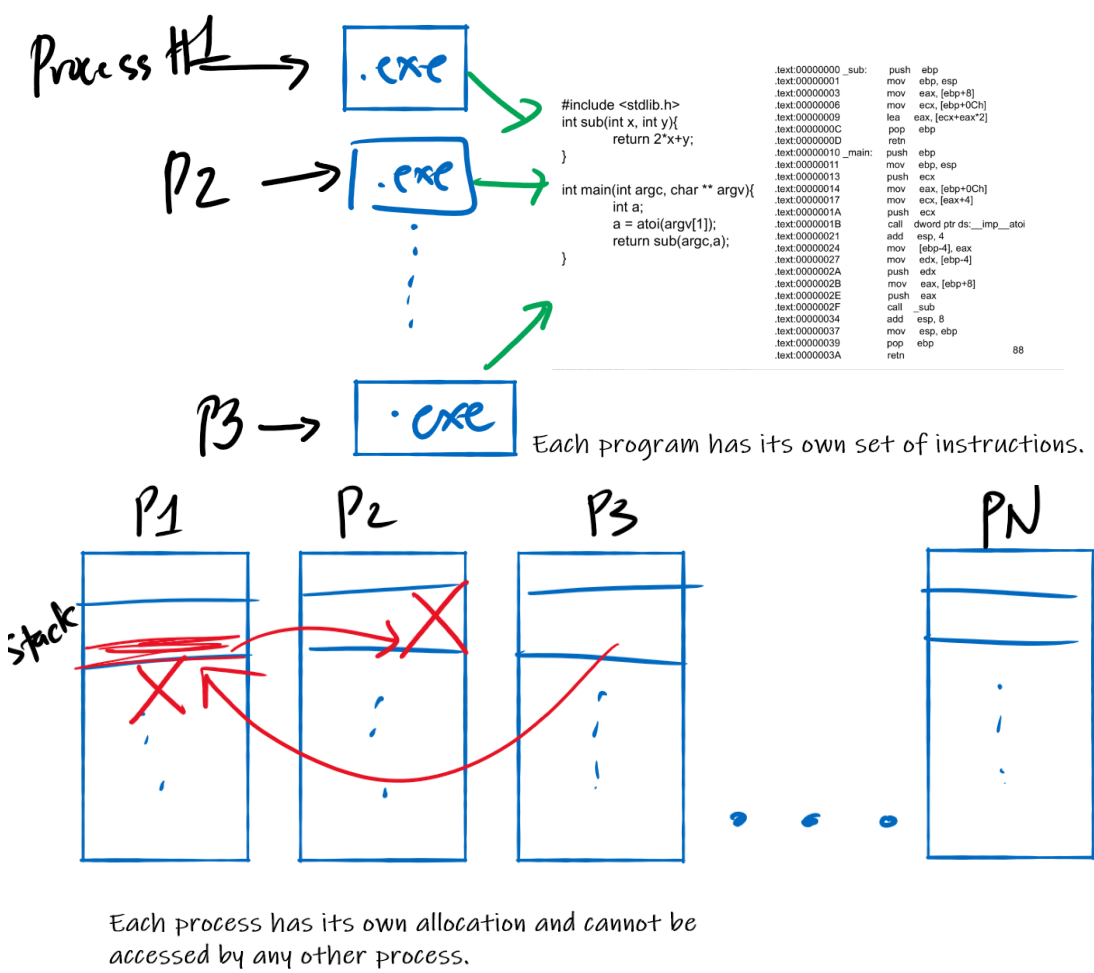


Components of a Program

- Main Program – points to the main function
- Subroutine – contains all functions declared in the code
- Libraries – external functions or dependencies required
- Symbol table – holds variables
- Stack – holds temporary variables for function calls. Also stores the return address of a given function call.
- Heap – memory for dynamically allocated objects.

Program components	Real-world	Remarks	Machine project/OS emulator	Remarks
Main program	✓		✓	
Subroutine	✓	User-defined functions (e.g. creating factorial function)	✓	User-defined instructions like ADD, READ, WRITE are analogous to subroutines in real-world OS.
Libraries	✓	Programs often link to external libraries for	✗	The OS emulator does not explicitly support external libraries or

		extended functionality and dependencies		dependencies for processes.
Symbol table	✓	The symbol table holds variables and other symbols, mapping them to memory locations.	✓	The OS emulator explicitly states that "uint16 variables are stored in the symbol table segment of the process".
Stack	✓	The stack is used for temporary variables, function call arguments, and return addresses.	X	
Heap	✓	The heap is used for dynamic memory allocation during program execution.	✓	Read and write instructions operate on memory addresses where values remain persistent. There must be a dedicated memory space.



In the operating system, each process is represented by a **Process Control Block (PCB)**. A typical PCB contains:

- Process State
- Program Counter
- CPU Registers
- CPU-Scheduling Information
- Memory-Management Information

- Accounting Information
- I/O Status Information

C++ Representation of a Process

In our emulator, we represent the PCB using a `Process` class. This class manages the process lifecycle, its instructions, and its local variables.

```
### Proposed `Process.h`
```cpp
#pragma once
#include <memory>
#include <vector>
#include <string>
#include "ICommand.h"
#include "SymbolTable.h"

class Process {
public:
 enum ProcessState {
 READY,
 RUNNING,
 WAITING,
 FINISHED
 };

 Process(int pid, std::string name);

 void addCommand(std::shared_ptr<ICommand> command);
 void executeCurrentCommand();
 void moveToNextLine();

 bool isFinished() const;
 int getPID() const;
 ProcessState getState() const;
 std::string getName() const;

 // Symbol Table access
 SymbolTable& getSymbolTable();

private:
 int pid;
 std::string name;
 ProcessState currentState;
 int commandCounter;

 std::vector<std::shared_ptr<ICommand>> commandList;
 SymbolTable symbolTable;
};
```
```

xv6 struct proc Expansion

In xv6, the struct proc serves as the Process Control Block. Below is the definition from the xv6-riscv version:

```
// Per-process state
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
```

```

enum procstate state;          // Process state
void *chan;                   // If non-zero, sleeping on chan
int killed;                   // If non-zero, have been killed
int xstate;                   // Exit status to be returned to parent's wait
int pid;                      // Process ID

// wait_lock must be held when using this:
struct proc *parent;          // Parent process

// these are private to the process, so p->lock need not be held.
uint64 kstack;                // Virtual address of kernel stack
uint64 sz;                    // size of process memory (bytes)
pagetable_t pagetable;        // Pythecal page table
struct trapframe *trapframe;  // data page for trampoline.S
struct context context;        // swtch() here to run process
struct file *ofile[NOFILE];   // Open files
struct inode *cwd;            // Current directory
char name[16];                // Process name (debugging)
};

```

Why is spinlock needed in struct proc?

In a multiprocessor system like xv6, multiple CPUs may attempt to access or modify the same process simultaneously. The spinlock protects the integrity of the process's state and other fields.

- **State Atomicity:** Prevents race conditions where two CPUs might try to schedule or wake up the same process at the same time.
- **Critical Sections:** Functions like `yield()`, `sleep()`, and `wakeup()` acquire the `p->lock` to ensure that state transitions (e.g., from `RUNNING` to `SLEEPING`) happen atomically without being interrupted or observed in an inconsistent state by the scheduler.

Key Fields:

- **state:** Defines the current lifecycle of the process. In xv6, these are defined in `enum procstate`:
- `enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };`
- **sz:** Tracks the size of the process memory in bytes, which is critical for memory management and allocation.
- **pid:** A unique identifier for the process, used by the kernel to manage and schedule it.
- **context:** Contains the register state (like `ra`, `sp`, `s0-s11`) used by the scheduler to perform context switching.
- **pagetable:** Points to the process's private page table, ensuring memory isolation between processes.

Representing process instructions

The general idea is to create a command/instruction interface that all other instructions will implement. E.g., `print`, `add`, `read`, `write`, etc.

Below is a simple example of how to implement a `print` command, without variable assignments:

```

5  class ICommand
6  {
7  public:
8      enum CommandType
9      {
10         IO,
11         PRINT
12     };
13
14     ICommand(int pid, CommandType commandType);
15     CommandType getCommandType();
16     virtual void execute();
17
18 protected:
19     int pid;
20     CommandType commandType;
21 };
22
23 inline ICommand::CommandType ICommand::getCommandType()
24 {
25     return this->commandType;
26 }
27
28 inline void ICommand::execute()
29 {
30     IETThread::sleep(ms::Delay::PER_INSTRUCTION_DELAY);
31 }
32
33 inline ICommand::ICommand(int pid, CommandType commandType)
34 {
35     this->pid = pid;
36     this->commandType = commandType;
37 }

```

```

4  class PrintCommand : public ICommand
5  {
6  public:
7      PrintCommand(int pid, String& toPrint);
8      void execute() override;
9  private:
10     String toPrint;
11 };

```

```

5  PrintCommand::PrintCommand(int pid, String& toPrint) : ICommand(pid, CommandType::PRINT)
6  {
7      this->toPrint = toPrint;
8  }
9
10 void PrintCommand::execute()
11 {
12     ICommand::execute();
13
14     // std::cout << "PID " << this->pid << ":" << this->toPrint << std::endl;
15     std::stringstream msg; msg << String(Ptr: "PID ") << this->pid << ":" << this->toPrint << std::endl;
16     MessageBuffer::log(message: msg.str());
17 }

```

Rudimentary example of a variable declaration command

```

712 class DeclareCommand : public ICommand
713 {
714 public:
715     // Constructor for DECLARE: takes variable name and initial value
716     DeclareCommand(int pid, String& varName, uint16_t value) : ICommand(pid, CommandType::DECLARE)
717     {
718         this->varName = varName;
719         this->value = value;
720         //IMPORTANT: Variables are only stored temporarily. To make them "appear" or be associated in the process,
721         //the actual declaration must occur in the execute() command.
722     }
723
724     void execute() override;
725
726     //TODO: The actual declaration --> symbol table association + placement in memory.
727     //NOTE: Consider using std::move or move-based operation for transferring the data without copying.
728     void performDeclaration();
729
730 private:
731     String varName;
732     uint16_t value; // uint16_t for the value
733 };
734
735 void DeclareCommand::execute()
736 {
737     ICommand::execute(); // Call base class execute() for common behavior
738
739     performDeclaration(); //perform the actual declaration and storing in symbol table + memory.
740
741     std::stringstream msg;
742     msg << "Executing DECLARE command for PID " << this->pid
743     | << ": variable '" << this->varName << "' with value " << this->value << std::endl;
744     std::cout << msg.str();
745 }

```

Activities:

- Redesign the print command such that it can accept and print one variable as stated in the specs.
- Design the variable declaration command.
- Design the remaining commands: ADD, SUBTRACT, SLEEP, FOR. For each command, show the header implementation and provide a documentation/report discussing your planned implementation.

Symbol Table

A process needs to store and retrieve variables during execution. The **Symbol Table** maps variable names (strings) to their values (e.g., integers).

Proposed SymbolTable.h

```
#pragma once
#include <string>
#include <unordered_map>
#include <cstdint>

class SymbolTable {
public:
    void setVariable(const std::string& name, int value) {
        table[name] = value;
    }

    int getVariable(const std::string& name) {
        if (table.find(name) != table.end()) {
            return table[name];
        }
        return 0; // Default or error value
    }

    bool hasVariable(const std::string& name) const {
        return table.find(name) != table.end();
    }

private:
    std::unordered_map<std::string, int> table;
};
```

Note: Where is the Symbol Table in xv6?

In the C++ Emulator, we use an explicit SymbolTable class (or std::unordered_map) stored in the PCB to track variables. In **xv6 (and real operating systems)**, the "Symbol Table" does not exist inside the PCB at runtime.

- **During Compilation:** The compiler/linker creates a symbol table and stores it in the **ELF executable file** (e.g., in .symtab or .strtab sections). This maps human-readable names to memory addresses.
- **At Runtime:** The kernel is "symbol-blind." It only sees memory addresses and values. All variables reside in the **Process Address Space** (specifically in the Data segment for globals or the Stack for local variables), which are managed via the pagetable.
- **Debugging:** Tools like gdb or the xv6 serial console use the symbols from the binary file to map addresses back to names for the developer's convenience.

CPU Scheduling

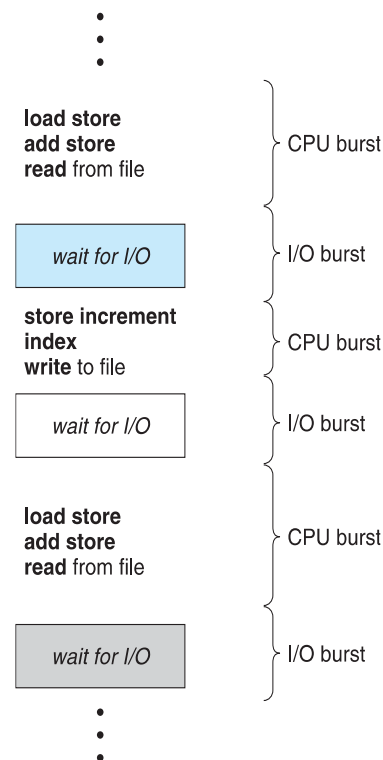
- CPU scheduling is a subset of process scheduling.
 - Process scheduling deals with the overall lifecycle and interactions of processes → processes may not always require the CPU. E.g. GPU rendering, I/O operation.
 - CPU scheduling specifically deals with deciding which process to execute next on the central processing unit (CPU). It involves selecting a process from the ready queue and allocating CPU time to that process.
 - However, all processes need the CPU at least once in their lifetime → for a process to be scheduled on the GPU, it must first have a CPU instruction requesting the transfer of data and GPU-specific instructions into the GPU.
- We will primarily be tackling CPU scheduling algorithms.

Objective of CPU scheduling:

The primary goal of CPU scheduling is to maximize CPU utilization and throughput while minimizing waiting times and response times for processes.

Background:

- Multiple processes compete for CPU time, and CPU scheduling determines which process to execute next.
- When the operating system switches from executing one process to another, it performs a context switch. This involves saving the current process's state and restoring the next process's state.
- Various scheduling algorithms exist to determine the order in which processes are executed. Common algorithms include First-Come, First-Served (FCFS), Shortest Job Next (SJN), Priority Scheduling, Round Robin, and Multilevel Queue Scheduling.
- A CPU can be on idle state such as the following case on the right. Program has “scattered” sections of I/O-related code. Whenever there’s an I/O code, the process must be removed from the CPU and move to the device queue (and gets executed in the I/O system program).
- A commercial OS will never have long CPU idle times even if the user is not seeing anything on-screen.



Terms

- CPU – not the physical CPU hardware. A logical unit for executing a process.
- Ready queue – holds processes that should be executed in the CPU.
- Short-term scheduler - selects from among the processes in ready queue and allocates to the CPU.
- Long-term scheduler – decides which processes should go first to the short-term scheduler. NOTE: Not all OS have this.
- CPU scheduling decisions may take place when a process:
 1. Switches from running to waiting state
 2. Switches from running to ready state
 3. Switches from waiting to ready
 4. Terminates

Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – # of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process
- **Waiting time** – amount of time a process has been waiting in the ready queue

- **Response time** - amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

The **CPU Scheduling Problem** is formally defined as:

Given a set of processes $P = \{P_1, P_2, \dots, P_N\}$ where $P_i \in P$ contains $\{A, C, X/\text{null}\}$ where:

- A = arrival time C = CPU time X = priority

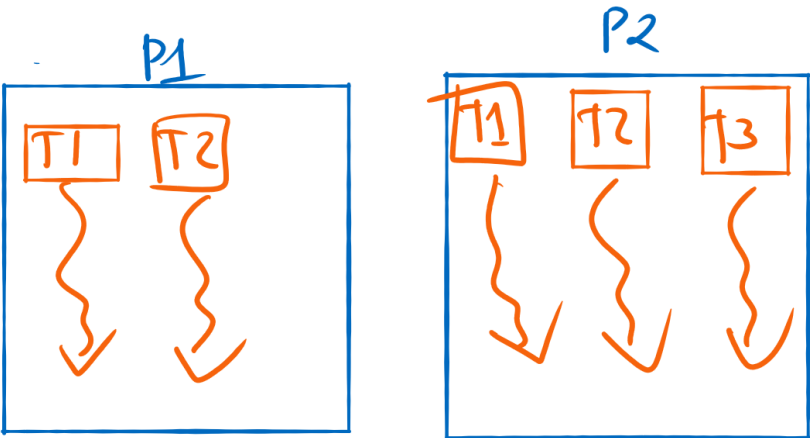
and a **CPU scheduling algorithm**, find O which is a set of ordered P according to sequence of execution.

CPU Scheduling Algorithm Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

INTRODUCTION TO THREADS

- Thread is a lightweight process. It allows us to parallelize certain logical operations.
- Contextualizing this to our marquee console, we can separate the marquee drawing from the keyboard polling.



- A thread is a lightweight process
- A sequence of instructions to execute in parallel with another thread.
- Since it is within a process, memory is shared.
- Multiple threads can access the same data structure, functions, etc.

Hands-On Activity: Create your first “Hello World” thread in C++.

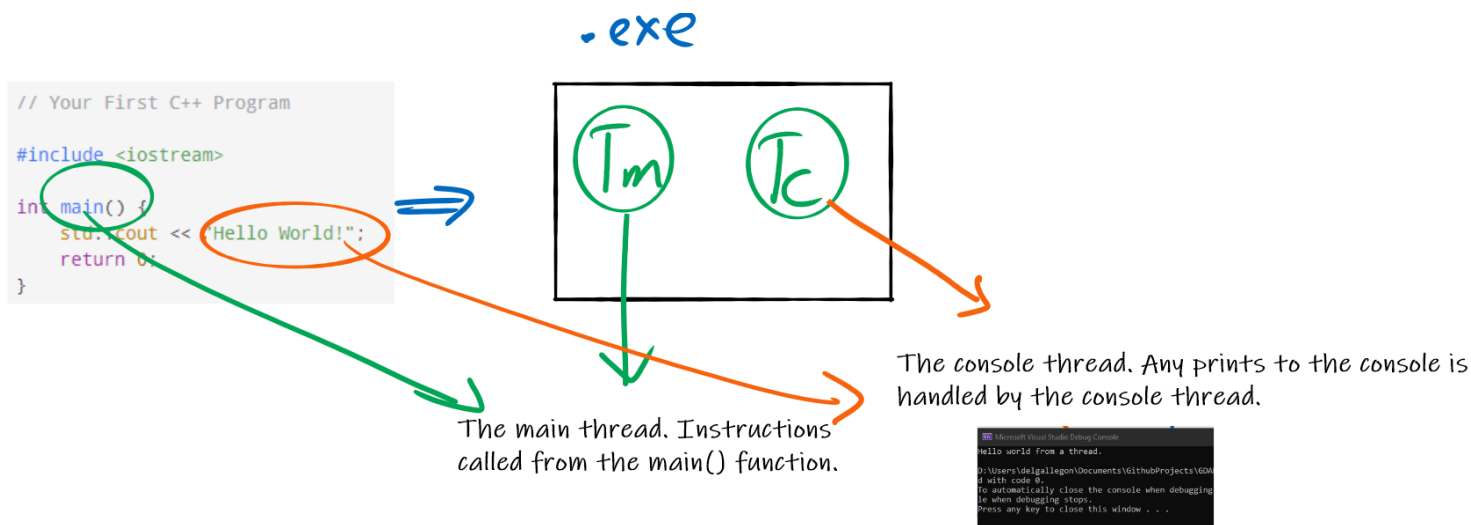
```
19 void testFunctionThread()
20 {
21     std::cout << "Hello world from a thread. " << std::endl;
22 }
23
24 int main() {
25     //createHWThreads();
26     std::thread myThread(testFunctionThread);
27 }
```

NOTE: Running the above program will trigger a runtime error. C++ has a guarding mechanism that should force the developer to call either join() or detach().

```
24 int main() {
25     //createHWThreads();
26     std::thread myThread(testFunctionThread);
27     myThread.join();
28 }
```

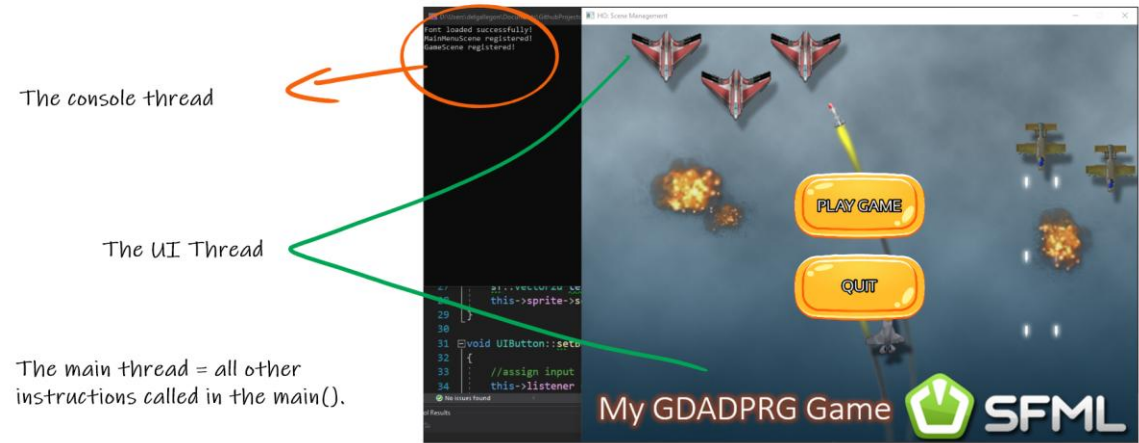
A Windows program - Default thread handling.

A default Windows program always has the following set of threads: main thread, UI thread. See illustration below.



The console thread persistently runs until the main program is closed.

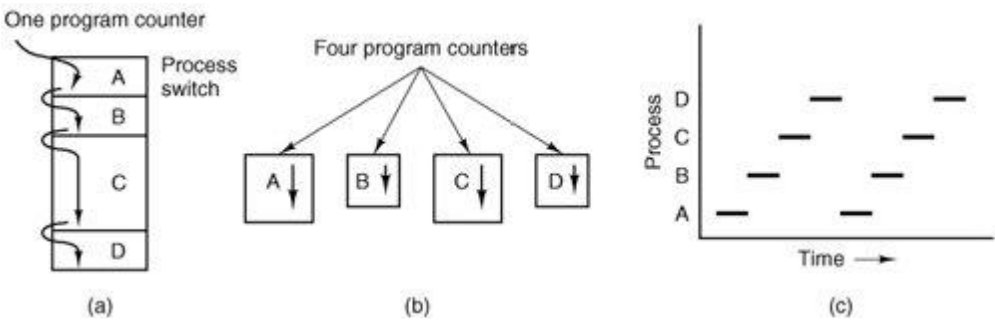
For programs that has a Window screen, such as game applications, there are actually 3 threads by default.



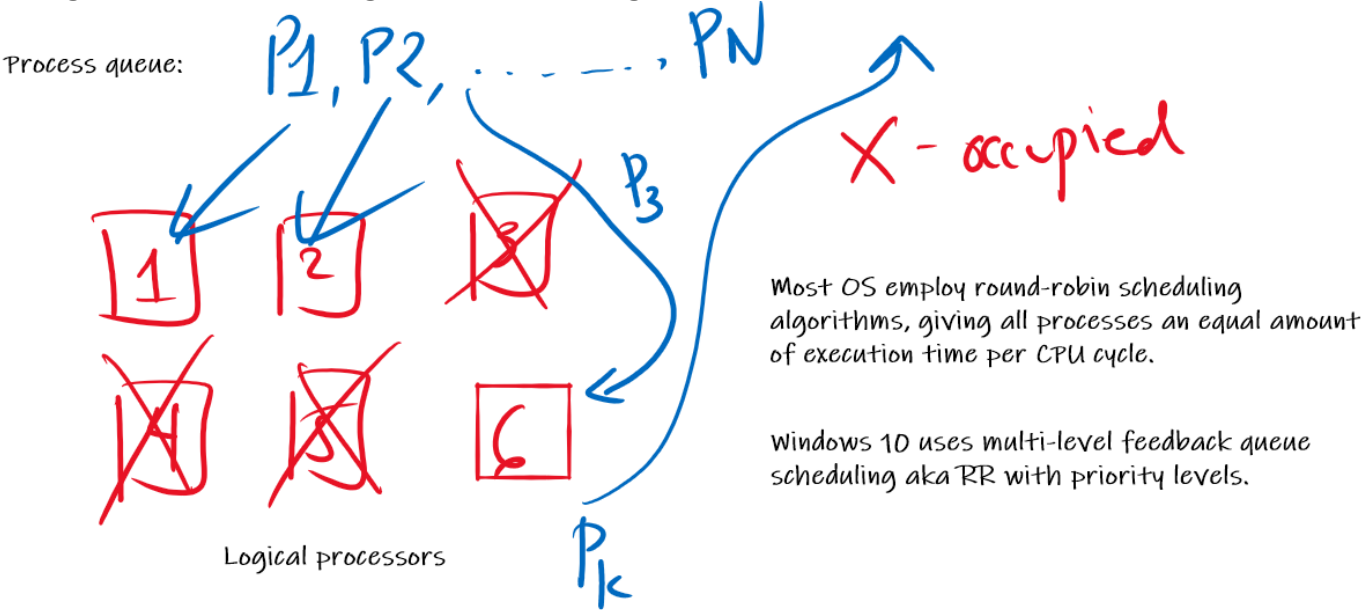
NOTE: All the spawned threads are independent from one another. Information is being passed to each thread through **message passing** (e.g. `std::cout`, `render/draw` frame in SFML/OPENGL).

TODO: Show a walkthrough of how a worker thread handles the marquee implementation: `MarqueeWorkerThread`.

Illustration of multiprocessing using a context-switching single-core processor vs multi-core processor



A diagram demonstrating CPU scheduling



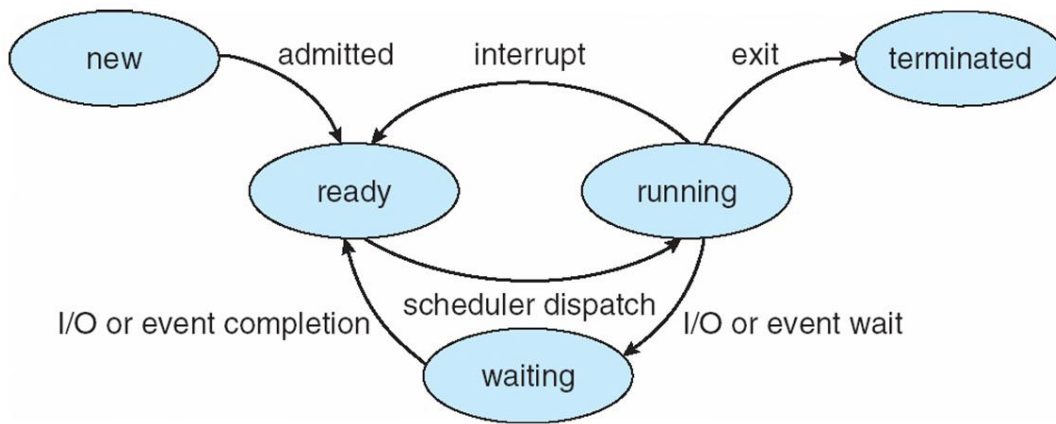
Process transition

- A program is in **passive** state when it is stored on disk (as an executable file).
- A program becomes a process when the executable file is loaded into memory (**active** state).
- Execution of program started via GUI mouse clicks, command line entry of its name, etc.
- One program can have several processes. E.g. consider a user opening a program multiple times.

Process states

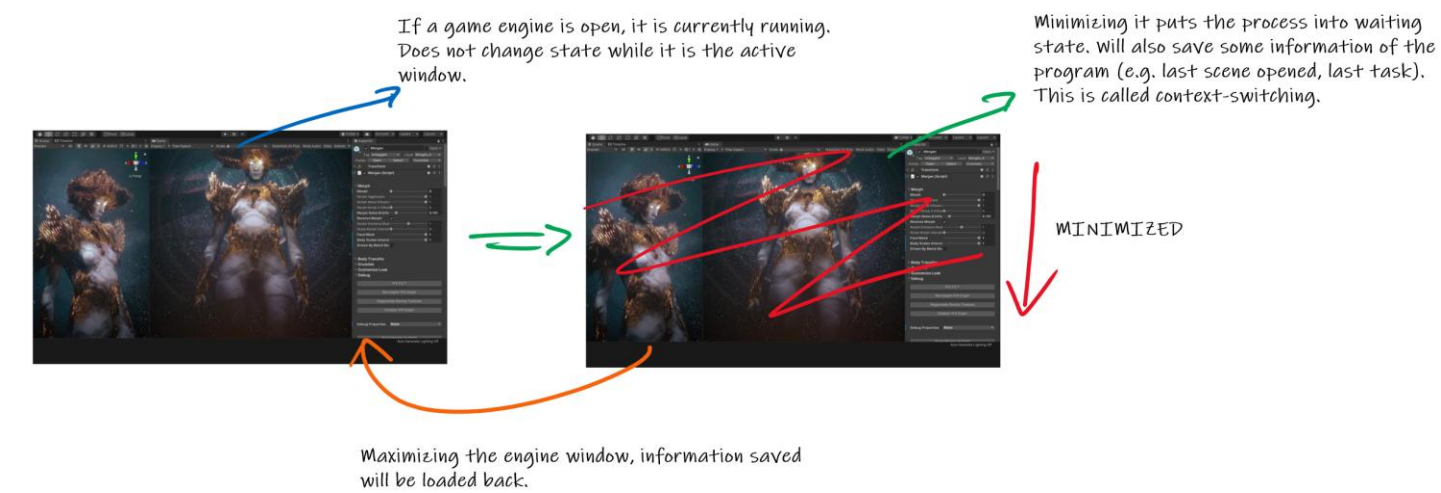
As a process executes, it changes **state**

- **new**: The process is being created
- **running**: Instructions are being executed
- **waiting**: The process is waiting for some event to occur
- **ready**: The process is waiting to be assigned to a processor
- **terminated**: The process has finished execution



Very similar to application lifecycle for mobile.

Processes can move to different states frequently. Consider this example:



OBSERVATION: Any active window implies that the process does not change state frequently.

| Background processes (147) | | |
|--------------------------------------|------|----------|
| > Adobe Acrobat Update Service (...) | 0% | 0.3 MB |
| Adobe Collaboration Synchroni... | 0% | 1.5 MB |
| Adobe Collaboration Synchroni... | 0% | 0.7 MB |
| Adobe RdrcEF (32 bit) | 0% | 1.4 MB |
| Adobe RdrcEF (32 bit) | 0% | 5.9 MB |
| AlSuite3.exe (32 bit) | 0% | 0.4 MB |
| > Antimalware Service Executable | 2.1% | 193.9 MB |
| Application Frame Host | 0.5% | 10.9 MB |
| > AsSysCtrlService.exe (32 bit) | 0% | 0.3 MB |
| > ASUS Com Service (32 bit) | 0% | 0.5 MB |
| > ASUS Motherboard Fan Control ... | 0% | 0.7 MB |
| AsusUpdateCheck.exe | 0% | 0.6 MB |
| > Calculator (2) | 0% | 0.4 MB |

However, background processes, such as services, move to various states frequently.

REASON: Foreground processes has more priority, a background process may stop execution at any time when additional resources are needed.

Hands-On Activity: Create your first “Hello World” thread in C++.

```
19 void testFunctionThread()
20 {
21     std::cout << "Hello world from a thread. " << std::endl;
22 }
23
24 int main() {
25     //createHWThreads();
26     std::thread myThread(testFunctionThread);
27 }
```

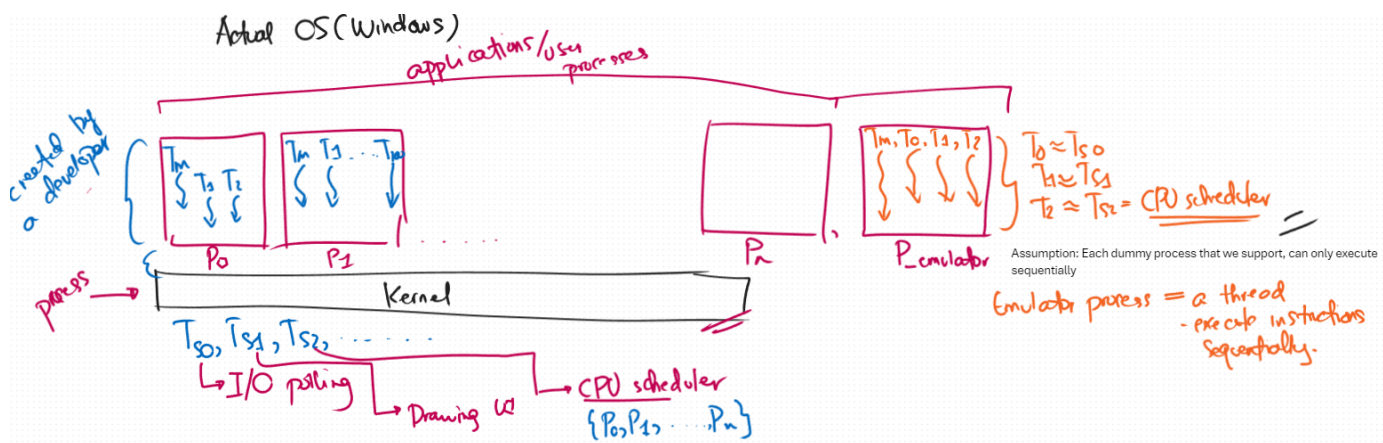
NOTE: Running the above program will trigger a runtime error. C++ has a guarding mechanism that should force the developer to call either join() or detach().

```
24 int main() {
25     //createHWThreads();
26     std::thread myThread(testFunctionThread);
27     myThread.join();
28 }
```

For our CSOPESY emulator, we will represent processes, as threads. Why?

- The application itself can be viewed as a box of your own PC, where we simulate how OS behaves.

- Processes use our imaginary resources → threads provide us easy access to these resources.

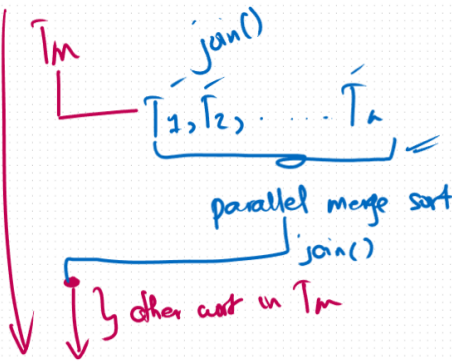
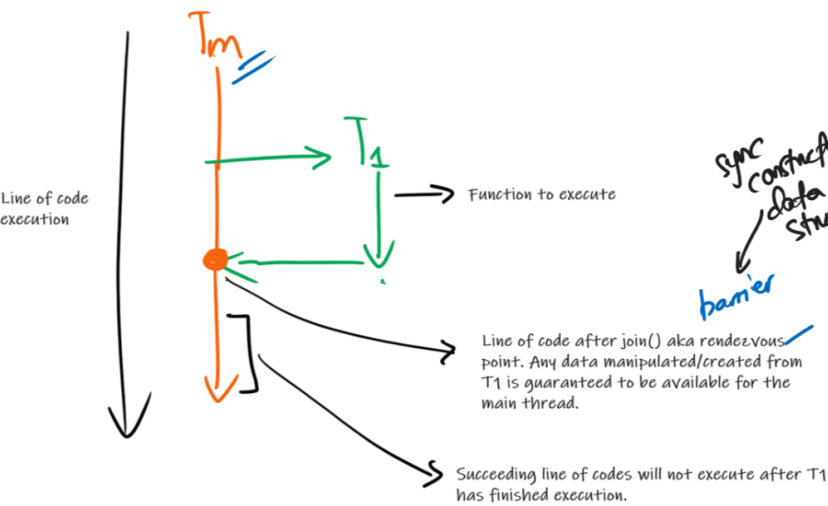


using `<thread.h>`

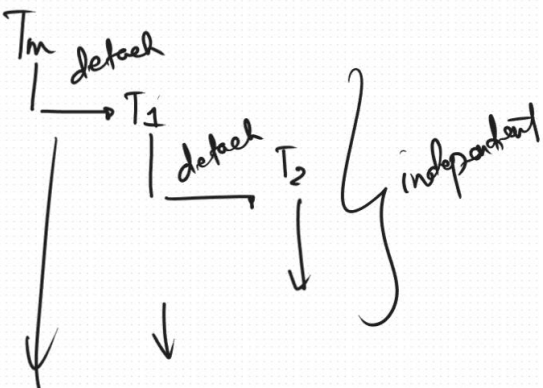
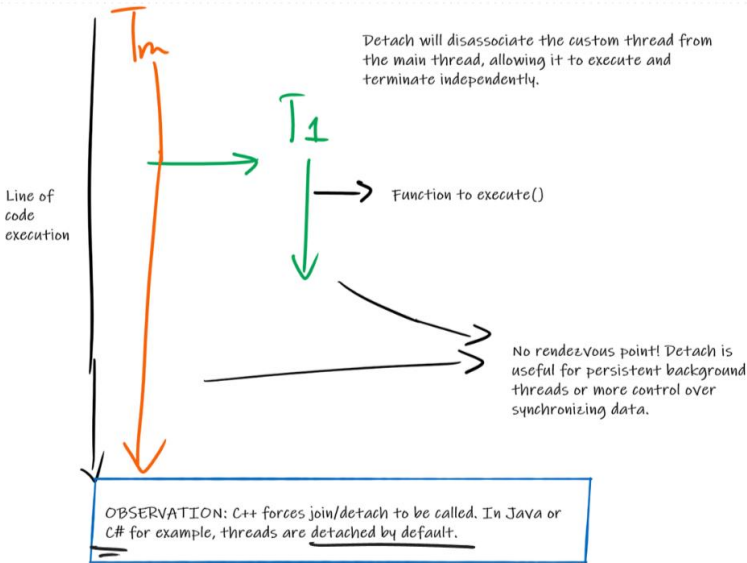
```
24 int main() {
25     //createHWThreads();
26     std::thread myThread(testFunctionThread);
27     myThread.join();
28 }
```

Based on C++'s compiler design, all threads must call either `join()` or `detach()`

Based on C++'s compiler design, all threads must call either `join()` or `detach()`.



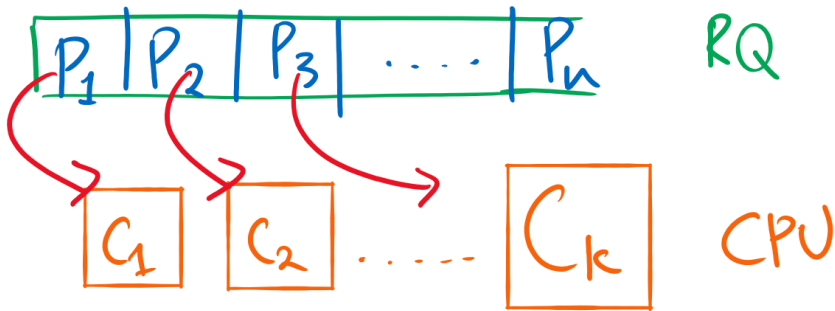
Detach



Demonstration of CPU Scheduling: Shortest-Job-First (Pre-Emptive)

Shortest-job-first, by the name itself, selects processes with the shortest CPU burst for scheduling. Consider the following example:

The ready queue holds pending processes



Each logical core can hold one process for execution.

When the process arrives at the ready queue

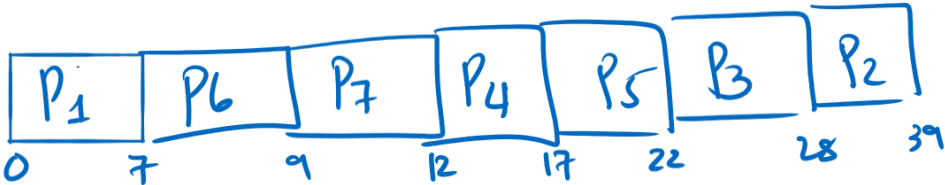
Total CPU time needed to finish it

| Process | Arrival Time | CPU Burst |
|---------|--------------|-----------|
| P1 | 0 | 7 |
| P2 | 1 | 11 |
| P3 | 2 | 6 |
| P4 | 3 | 5 |
| P5 | 4 | 5 |
| P6 | 6 | 2 |
| P7 | 7 | 3 |

Using shortest-job-first (pre-emptive)

RQ: P_2 P_3 P_4 P_5 P_6 P_7

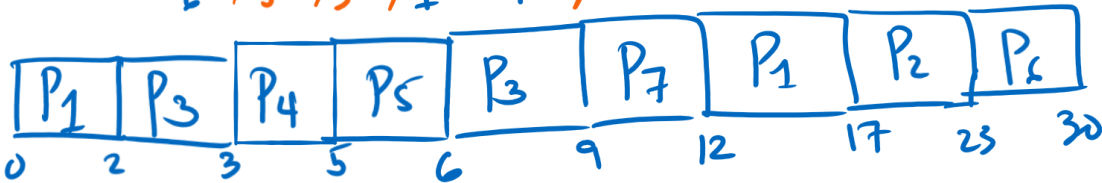
Gantt chart of CPU execution



To show how pre-emptive works

| Process | Arrival Time | CPU Burst |
|---------|--------------|-----------------|
| P1 | 0 | 7 |
| P2 | 1 | 11 6 |
| P3 | 2 | 6 4 |
| P4 | 3 | 5 2 |
| P5 | 4 | 5 1 |
| P6 | 6 | 2 7 |
| P7 | 7 | 3 |

RQ: P_2 P_4 P_3 P_5 P_6 P_7



Algorithmic Overview of Pre-Emptive SJF

Inputs

- List of processes inside the ready queue, R
- Given a set of processes $P = \{P_1, P_2, \dots, P_N\}$ where $P_i \in P$ contains $\{A, C, X/\text{null}\}$ where:
 - A = arrival time
 - C = CPU time
 - X = priority

Algorithm:

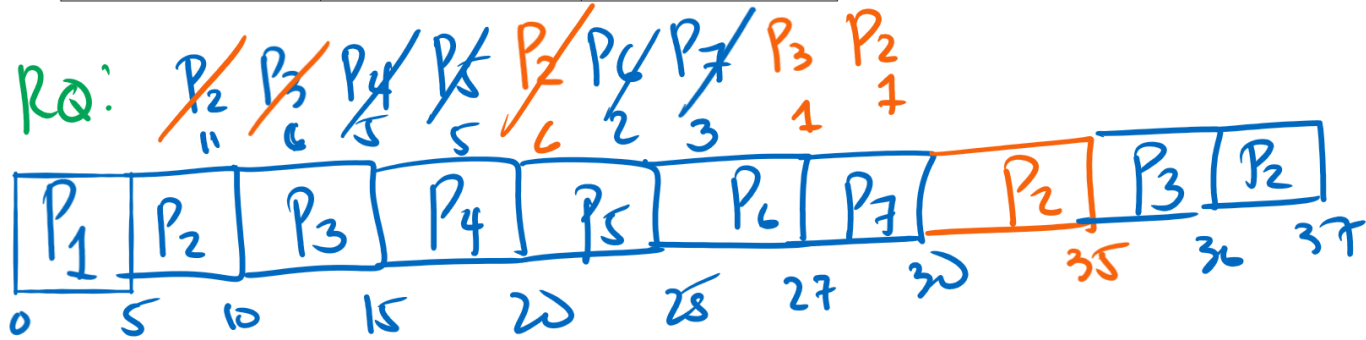
- For each CPU cycle, do the following:
 - Update R if there's any pending process to be scheduled.
 - If applicable, sort R where the first element satisfies $P_0 = \min(C)$.
 - Select $P_{\text{candidate}}$, such that $P_{\text{candidate}} = \min(C)$
 - Execute $P_{\text{candidate}}$. $P_{\text{candidate}}, C--$;

Demonstration of CPU Scheduling: Round-Robin

Promotes fairness policy by cycling through all processes that arrive evenly. Each process receives a fair amount of CPU time for execution.

Using round-robin with CPU burst allocation of 5

| Process | Arrival Time | CPU Burst |
|---------|--------------|-----------|
| P1 | 0 | 7 |
| P2 | 1 | 11 |
| P3 | 2 | 6 |
| P4 | 3 | 5 |
| P5 | 4 | 5 |
| P6 | 6 | 2 |
| P7 | 7 | 3 |



IDEA: Promote fairness on all processes as much as possible

Algorithmic Overview of Round-Robin

Inputs

- List of processes inside the ready queue, R
- Time quantum slice, T.
- Given a set of processes $P = \{P_1, P_2, \dots, P_N\}$ where $P_i \in P$ contains $\{A, C, X/\text{null}\}$ where:
 - A = arrival time
 - C = CPU cycles consumed. Starts with $C = 0$.
 - X = priority

Algorithm:

- For each CPU cycle, do the following:

- a. Update R if there's any pending process to be scheduled.
- b. Select the first process in R (or randomly) to be the $P_{candidate}$
- c. Execute $P_{candidate} \cdot P_{candidate}, C++;$
- d. If $P_{candidate}, C = T$, then perform #b. Put $P_{candidate}$ at the end of R. Otherwise, perform #c.

Multi-Processor Scheduling

- When there are multiple cores/logical compute units present, two possible scheduling strategies could be:
 - All processes may be in a common ready queue
 - Each processor may have its own private queue of threads
- This approach is called **symmetric multiprocessing (SMP)**. The other approach is asymmetric multiprocessing.
 - CPU scheduling in a multiprocessor system has all scheduling decisions, I/O processing, and other system activities handled by a single processor – the master server.
 - This asymmetric multiprocessing is simple because only one core accesses the system data structures, reducing the need for data sharing. The downfall of this approach is the master server becomes a potential bottleneck where overall system performance may be reduced.

Illustration of SMP vs AMP is shown below:

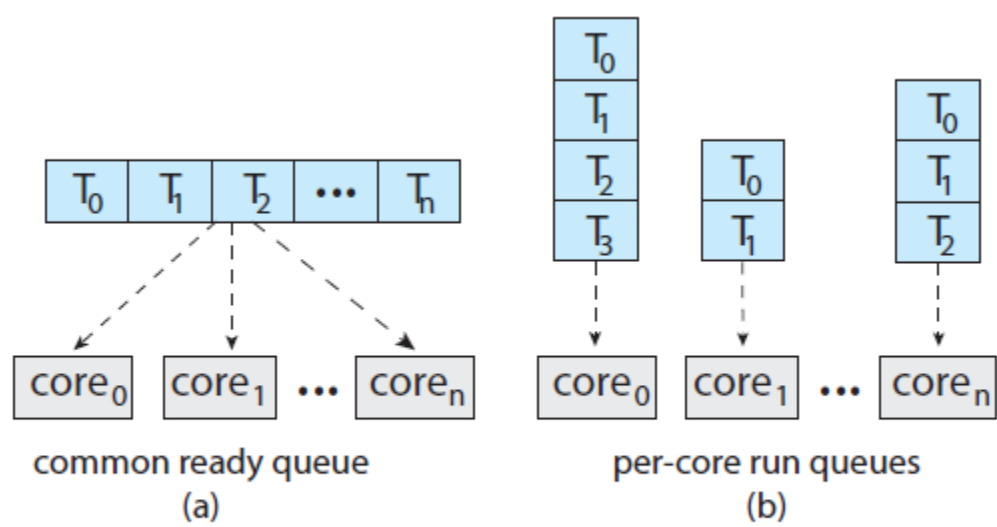
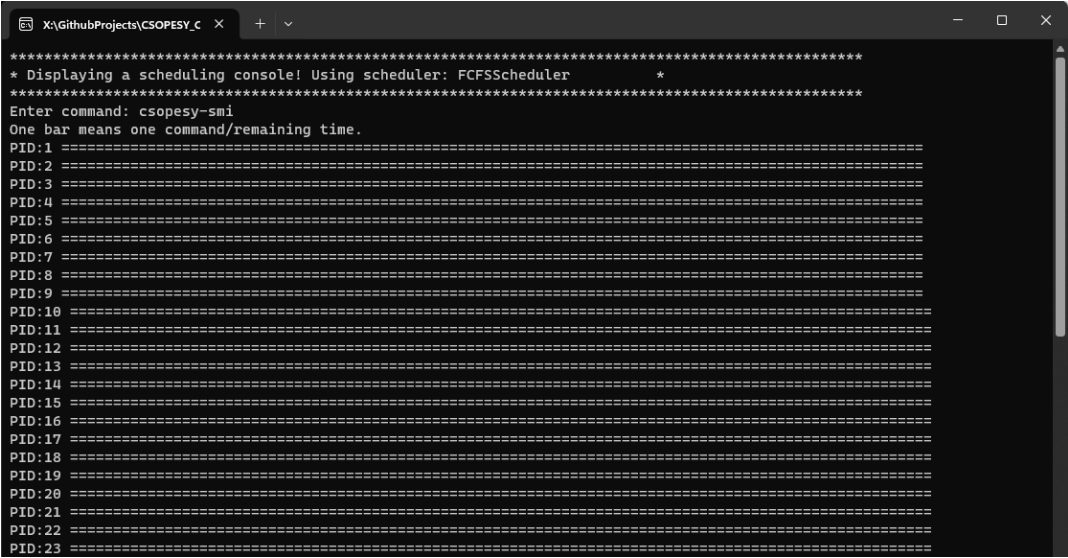


Figure 5.11 Organization of ready queues.

Emulating a CPU scheduler

- We need the following: process representation, scheduler representation, and a way to debug/verify the correctness of our chosen scheduling algorithm.

The UI/frontend could first look like this:



OR

```
25  _____
26  |  CSOPESY  |
27  |_____|
28
29  -----
30  Welcome to CSOPESY Emulator!
31
32  Developers:
33  Del Gallego, Neil Patrick
34
35  Last updated: 01-18-2024
36
37  root:\> screen -ls
38  CPU utilization: 100%
39  Cores used: 4
40  Cores available: 0
41
42  -----
43  Running processes:
44  process05 (01/18/2024 09:15:22AM) Core: 0 1235 / 5876
45  process06 (01/18/2024 09:17:22AM) Core: 1 3 / 5876
46  process07 (01/18/2024 09:17:45AM) Core: 2 9 / 1000
47  process08 (01/18/2024 09:18:58AM) Core: 3 12 / 80
48
49  Finished processes:
50  process01 (01/18/2024 09:00:21AM) Finished 5876 / 5876
51  process02 (01/18/2024 09:00:22AM) Finished 5876 / 5876
52  process03 (01/18/2024 09:00:42AM) Finished 1000 / 1000
53  process04 (01/18/2024 09:00:53AM) Finished 80 / 80
54  -----
55
56  root:\> report-util
57  root:\> Report generated at C:/csopesy-log.txt!
```

The user could always type “process-smi” to view the gantt chart. When all processes are finished, “csopesy-smi” should now display a summary of the ordering of processes executed and its **turnaround time**.

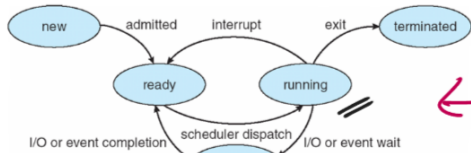
A representation of the process class:

```
556 //a template for the process
557 #include <iostream>
558 #include <string>
559 #include <cstdlib> // for rand() function
560
561 class Process {
562 private:
563     std::string name;
564     int id;
565     int totalInstructions;
566     int remainingInstructions;
567
568 public:
569     Process(const std::string& processName, int processId, int numInstructions)
570         : name(processName), id(processId), totalInstructions(numInstructions), remainingInstructions(numInstructions) {}
571
572     // Execute one instruction of the process
573     void executeInstruction() {
574         if (remainingInstructions > 0) {
575             std::cout << "Executing instruction for Process " << id << ": " << name << "\n";
576             remainingInstructions--;
577         } else {
578             std::cout << "Process " << id << ": " << name << " has already finished.\n";
579         }
580     }
581
582     // Get the remaining number of instructions
583     int getRemainingInstructions() const {
584         return remainingInstructions;
585     }
586
587     // Check if the process has finished
588     bool hasFinished() const {
589         return remainingInstructions == 0;
590     }
591 };
592
593 int main() {
594     // Create a sample process
595     Process myProcess("SampleProcess", 1, 10);
596
597     // Execute instructions until the process finishes
598     while (!myProcess.hasFinished()) {
599         myProcess.executeInstruction();
600     }
601
602     std::cout << "Process has finished!\n";
603
604     return 0;
605 }
```

Process representation

From Week 6 - Process Scheduling

- Process states
- As a process executes, it changes state
- new: The process is being created
 - running: Instructions are being executed
 - waiting: The process is waiting for some event to occur
 - ready: The process is waiting to be assigned to a processor
 - terminated: The process has finished execution



CPU scheduling decisions take place

Process table (ready) | Process table (running) | (finished)

P₀
P₁
P₂
...

```
42 private:
43     int pid;
44     String name;
45     typedef std::vector<std::shared_ptr<Command>> CommandList;
46     CommandList commandList;
47
48     int commandCounter;
49     int cpuCoreID = -1;
```



Process

int main()

1
2
3
4

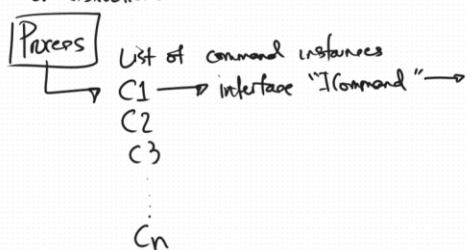
instructions/commands akin to machine code

program counter/current line of code

Page P₀, P₁, ..., P_n → P_{n-1} ← P₀

0
1
2
...
K

Context: Process will have N number of commands or instructions



P₀ → "print" x100
P₁ → "print" x1000

The process EXECUTES the list of commands in sequence.

Family of basic commands
→ simplest instructions of a process

- "PRINT"
- "I/O"
- "SLEEP"
- "ADD"
- "DN"

In real-world/actual OS, the list of commands is created by your compiler -- > discussed in a compiler theory course.

Source code → ^{compiler} → machine code

called by "CPU core worker"

```
30 void Process::executeCurrentCommand() const  
31 {  
32     this->commandList[this->commandCounter]-->execute();  
33 }  
34  
35 void Process::moveToNextLine()  
36 {  
37     this->commandCounter++;  
38 }
```

CPU scheduler (FCFS)

for RR, check the quantum slice remaining

Line 30
Line 35



CPU core worker

CPU workers are individual threads

A template for a first-come-first-serve scheduler:

```
608 //a template for an FCFS scheduler  
609 class FCFSScheduler {  
610 private:  
611     int numCores;  
612     std::vector<std::vector<Process>> processQueues; // One queue for each core  
613  
614 public:  
615     FCFSScheduler(int cores) : numCores(cores), processQueues(cores) {}  
616  
617     // Add a process to the scheduler  
618     void addProcess(const Process& process, int core = 0) {  
619         if (core >= 0 && core < numCores) {  
620             processQueues[core].push_back(process);  
621         } else {  
622             std::cerr << "Invalid core specified for process addition.\n";  
623         }  
624     }  
625  
626     // Sort the process queues based on remaining instructions (FCFS)  
627     void sortProcessQueues() {  
628         for (auto& queue : processQueues) {  
629             std::sort(queue.begin(), queue.end(), [](const Process& a, const Process& b) {  
630                 return a.getRemainingInstructions() > b.getRemainingInstructions();  
631             });  
632         }  
633     }  
634  
635     // Run the scheduler  
636     void runScheduler() {  
637         while (!processQueues[0].empty()) {  
638             for (int core = 0; core < numCores; ++core) {  
639                 if (!processQueues[core].empty()) {  
640                     Process currentProcess = processQueues[core].back();  
641                     processQueues[core].pop_back();  
642  
643                     while (!currentProcess.hasFinished()) {  
644                         currentProcess.executeInstruction();  
645                     }  
646  
647                     std::cout << "Process " << currentProcess.getRemainingInstructions() << " completed on Core " << core + 1 << ".\n";  
648                 }  
649             }  
650         }  
651     }  
652 };
```

The scheduler is designed to support multiple CPU cores that can be configured during compile-time.

Sample usage:

```
654 int main() {
655     // Seed the random number generator
656     std::srand(static_cast<unsigned>(std::time(0)));
657
658     // Assume the list of processes is given
659     std::vector<Process> processes;
660     processes.emplace_back("Process_1", 1, 10);
661     processes.emplace_back("Process_2", 2, 8);
662     processes.emplace_back("Process_3", 3, 15);
663
664     // Initialize FCFS Scheduler with 3 CPU cores
665     FCFSScheduler fcfsScheduler(3);
666
667     // Add processes to the scheduler
668     for (const auto& process : processes) {
669         fcfsScheduler.addProcess(process);
670     }
671
672     // Sort the process queues (FCFS)
673     fcfsScheduler.sortProcessQueues();
674
675     // Run the scheduler
676     fcfsScheduler.runScheduler();
677
678     return 0;
679 }
```

TODO: Discuss scheduler, process, and command implementations from the CSOPESY emulator.

Activity:

- Implement your schedulers in your emulator.
- Modify the scheduler such that the CPU core is actually a thread worker. If there are four cores, then processes get assigned to a "core", where its commands are executed.

Then, provide a function that can be called anytime from the main thread that writes down the CPU utilization with the following format:

| | Util (%) | Num processes in queue |
|--------|----------|------------------------|
| CPU 1: | 100 | 4 |
| CPU 2: | 100 | 5 |
| CPU 3: | 100 | 2 |
| CPU 4: | 100 | 6 |

The command can be "cpu-util."

- Design your scheduler in such a way that it allows additional processes to be scheduled during run-time.
 - Behavior à at startup, create 50 random processes.
 - Then add a command "add-process-set", that creates 50 random processes and schedule them in the ready queue.
 - Note that "cpu-util" command can be used at any time.

COMMON CPU SCHEDULING ALGORITHMS IN DETAIL

- There are two commonly used scheduling algorithms: round-robin (RR) and shortest-job-first (SJF)

Round-robin scheduling

- Round-robin scheduling is a preemptive scheduling algorithm used in operating systems.

- It is designed for time-sharing systems, where each process is assigned a fixed time slot or quantum.
- Key Features:
 - Time Slicing:
 - Each process is assigned a fixed time slice or time quantum during which it can execute.
 - Once a process exhausts its time quantum, it is moved to the back of the ready queue.
 - Preemption:
 - If a process doesn't finish within its time slice, it is preempted, and the next process in the queue is given a chance to execute.
- Queue Structure:
 - Processes are organized in a circular queue (hence the name "round-robin").
 - The scheduler traverses the queue in a circular fashion, giving each process an opportunity to execute in turn.
- Advantages:
 - Fairness:
 - Provides fairness in CPU allocation as each process gets an equal share of CPU time.
 - Low Latency:
 - Results in low process response time due to quick switching between processes.
 - Simple Implementation:
 - Implementation is relatively simple compared to some other scheduling algorithms.
- Disadvantages:
 - Convoy Effect:
 - Can lead to the convoy effect, where short processes are delayed by long processes waiting in the queue.
 - Poor Performance for I/O-Bound Processes:
 - Inefficient for I/O-bound processes as they may lose their time slice while waiting for I/O.
- Parameter:
 - The main parameter is the time quantum, which determines the maximum time a process can run before being preempted.
- Example:
 - Consider three processes A, B, and C with time slices of 20 ms each.
 - The scheduler allocates CPU time to each process in a circular manner:
A -> B -> C -> A -> B -> C -> ...

Use Cases

- Round-robin scheduling is suitable for systems where fairness and quick response time are crucial, such as interactive systems and time-sharing environments.

Dynamic Priority

- Some variants may adjust the priority of processes dynamically based on their behavior and resource needs.

Windows OS scheduler

Windows operating systems use a scheduling algorithm that incorporates elements of round-robin scheduling. In Windows, the scheduler uses a priority-based approach involving process time-sharing. While it's not a strict round-robin algorithm, the idea of allocating time slices to different processes and regularly switching between them is present.

Windows uses a priority-based, preemptive scheduling algorithm where each process is assigned a priority level. The scheduler assigns time slices (quantum) to processes based on their priority. Higher-priority processes get more frequent access to the CPU. If a process doesn't finish within its time slice, it is moved to the back of the queue, allowing other processes to execute.

While the Windows scheduler is more sophisticated and dynamic than a simple round-robin scheduler, the concept of time-sharing and fairness among processes is still a key part of its design. The Windows scheduler considers factors like process priority, thread priority, and process states to make intelligent scheduling decisions.

Shortest-job-first

- SJF is a non-preemptive or preemptive scheduling algorithm.
- Also known as Shortest Job Next (SJN) or Shortest Job Next.
- It selects the process that has the smallest total remaining processing time.
- Algorithm:
 - Processes are ordered by their total burst time (time required for execution).
 - The process with the shortest burst time is scheduled first.
 - If multiple processes have the same burst time, FCFS (First-Come-First-Serve) order is followed.
- Non-Preemptive SJF:
 - Once a process starts executing, it cannot be preempted until it completes.
 - Decisions are made only when a process completes or a new process arrives.
- Preemptive SJF:
 - Also known as Shortest Remaining Time First (SRTF).
 - A new process with a shorter burst time can preempt a currently running process.
 - This requires dynamic prioritization based on the remaining time.
- Advantages:
 - Minimizes waiting time and turnaround time.
 - Efficient for a system with predictable burst times.
- Disadvantages:
 - Requires knowledge of the total burst time, which is often not available in real-time. E.g. In Windows, all processes can run at any given time through user interactions à double-clicking on an icon.
 - Can lead to starvation for longer processes.
- Example Scenario:
 - Consider processes with burst times: P1(6), P2(8), P3(7), P4(3).
 - Non-Preemptive SJF would schedule: P4, P1, P3, P2.
- Note:
 - SJF is optimal for minimizing waiting time but requires accurate knowledge of process burst times, which may not be available in practice.

Multi-level queue scheduling

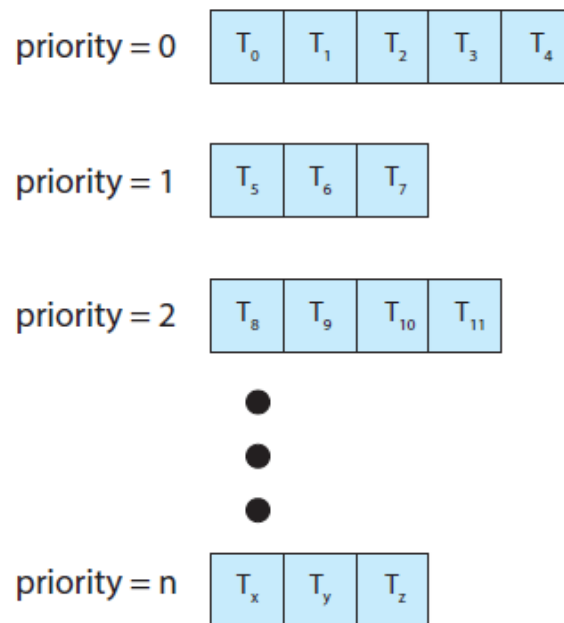


Figure 5.7 Separate queues for each priority.

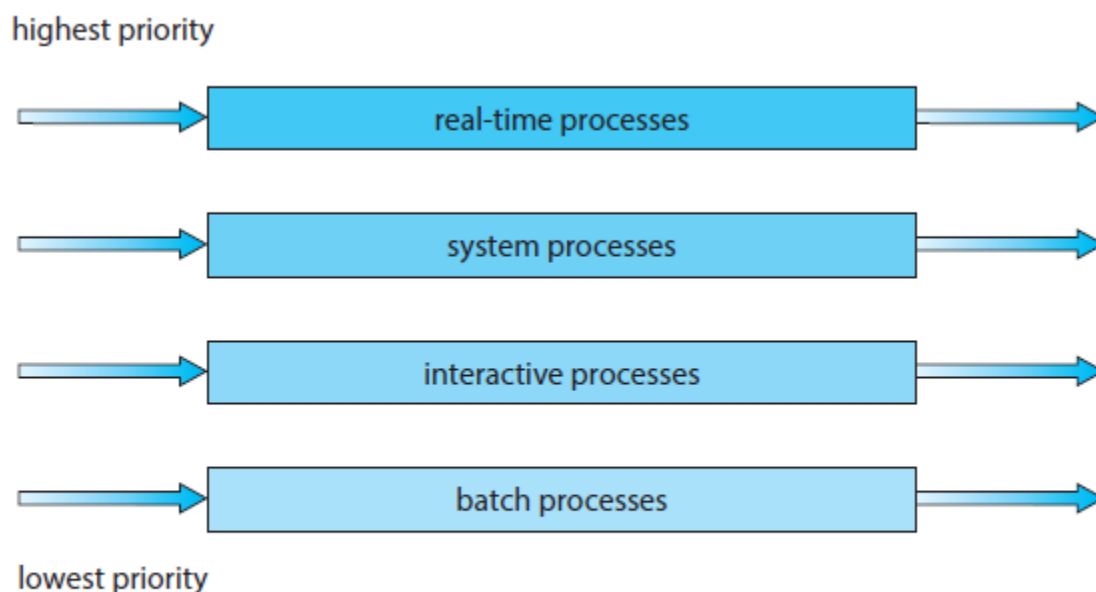


Figure 5.8 Multilevel queue scheduling.

- Overview:
 - Multi-Level Queue (MLQ) is a scheduling algorithm that divides processes into multiple queues based on priority.
 - Each queue can have its own scheduling algorithm.
 - Queues are assigned different priorities, and processes move between queues based on their characteristics.
- Key Components:
 - Multiple Queues:
 - Processes are assigned to one of several priority queues.
 - Each queue may have its own scheduling algorithm (e.g., FCFS, Round Robin).
 - Priority Assignment:
 - Queues are assigned different priorities.
 - Higher priority queues are processed before lower priority queues.
 - Queue Transition:
 - Processes can move between queues based on criteria (e.g., aging, priority adjustments).
 - Promotions and demotions based on process behavior.
- Algorithm Execution:

- **High-Priority Queue:**
 - Processes in the highest priority queue are scheduled first.
 - Processes may not be preempted within this queue.
- **Low-Priority Queue:**
 - Lower priority queues may use different scheduling algorithms.
 - Processes move to lower priority queues if they don't finish in the current queue.
- **Example Scenario:**
 - Consider three priority queues: High priority (Round Robin), Medium priority (FCFS), Low priority (SJF).
- **Advantages:**
 - Provides a way to manage processes with varying characteristics.
 - Better responsiveness for high-priority tasks.
- **Disadvantages:**
 - Requires careful tuning of priorities and transition rules.
 - May lead to starvation if not implemented properly.
- **Note:**
 - MLQ is a flexible approach that allows for different scheduling policies to be applied to different types of processes.

More information about Windows scheduler

Windows operating systems typically use a priority-based scheduling algorithm rather than a strict Multi-Level Queue (MLQ) scheduling approach. Windows uses a combination of priority levels, thread quantum, and a priority boost mechanism for thread scheduling.

In the Windows scheduling model:

- **Priority Levels:** Threads are assigned priority levels ranging from 0 to 31, with lower numbers indicating higher priority.
- **Quantum:** Each thread is given a time quantum during which it can execute. After the quantum expires, the thread may be moved to the back of the queue.
- **Priority Boost:** Windows incorporates a priority boost mechanism to prevent aging. If a thread waits too long for execution, it may receive a priority boost.

Activities:

- Implement some of your emulator's previously discussed scheduling algorithms, such as RR and SJF.
- Provide a design implementation for your multiprocessor scheduler following the diagrams provided below:

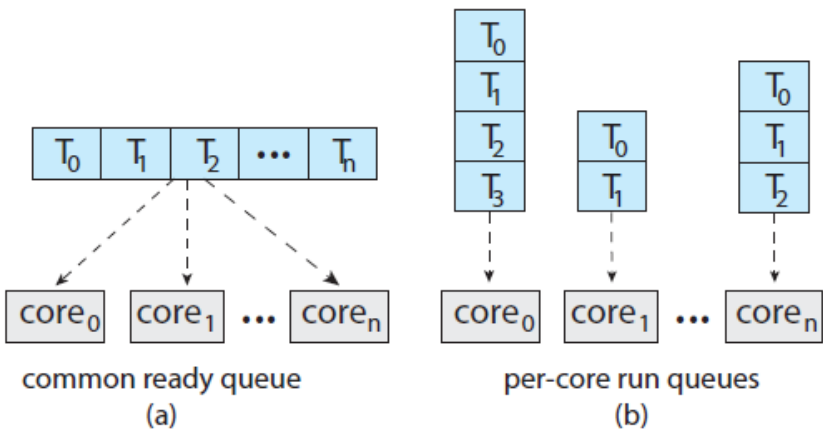


Figure 5.11 Organization of ready queues.

Design #1: There is only one ready queue.

Design #2: Ready queues are tied to CPU cores.